

Full-stack szoftverfejlesztés 2025/26/2-ZH2-L-A

A feladat megoldása során **semmilyen segédeszköz** használata **nem engedélyezett**.

A feladat megoldására fordítható idő **60 perc**.

A maximális pontszám 20, a sikeres teljesítéshez legalább 10 pontot kell elérnie.

Miről szól a feladat?

A feladat során egy pizzákat kezelő, REST API-ra épülő alkalmazáson fog dolgozni. Az alkalmazás kliens-szerver architektúrát valósít meg. A *server* mappában találja a szerver projektet, a *client*-ben pedig a klienst. Mindkét projektet localhoston kell majd futtatnia. Az adatréteget, CORS beállításokat, stílusokat a projekt már tartalmazza, ezek konfigurálásával nem kell foglalkoznia.

A megoldás során a kapott projekteket kell kiegészítenie. Az alábbi kép egy példát mutat a kliensre. Nem szükséges, hogy a saját megoldása ezzel pontosan megegyező legyen, ez csak iránymutatás. A funkcióknak azonban követniük kell a feladateleírást!

ID

3

Name

Sausage Stingray

Description

A special pizza made with spicy sausage and stingray pieces.

Price

26.99

Reset

Create pizza

Update pizza

Delete pizza

Pizzas

ID	Name	Description	Price
1	Buffalo Chicken	Spicy buffalo chicken, blue cheese crumbles, red onions, and mozzarella cheese on a tomato sauce base.	11.49€
2	Supreme	A loaded pizza with pepperoni, sausage, bell peppers, onions, mushrooms, olives, tomato sauce, and mozzarella cheese.	13.99€
3	Sausage Stingray	A special pizza made with spicy sausage and stingray pieces.	26.99€
4	Hawaii	A tropical pizza with ham, pineapple, and mozzarella cheese.	12.99€
5	Veggie Delight	A vegetarian pizza loaded with bell peppers, onions, mushrooms, olives, spinach, tomato sauce, and mozzarella cheese.	7.99€

Részletes leírás

Szerver

Állítsa be a szerver projekten, hogy az az 5566-os porton legyen elérhető a localhoston. (1p)

A szerver projektben hozzon létre egy *PizzaApiController* nevű controllert. (2p)

Ez *'pizzaapi'* néven legyen elérhető az url-ben. (3p)

Dependency injection és az *IPizzaServiceRepository* használatával tegye elérhetővé az adatréteg elérését. (1p)

Vegyen fel egy *GetPizzas* nevű végpontot, ami GET kérést szolgál ki és visszatérési értéként az adatrétegben tárolt összes pizzát visszaadja. (1p)

Vegyen fel egy *CreatePizza* nevű végpontot, ami POST kérést szolgál ki és létrehozza a paraméterként kapott pizzát. (1p)

Vegyen fel egy *DeletePizza* nevű végpontot, ami DELETE kérést szolgál ki és törli a paraméterként kapott pizzát. (1p)

Állítsa be a szerver projekten, hogy alpaértelmezetten a localhost:5566/pizzaapi url töltsön be a szerver indítását követően. (1p)

Kliens

Implementáljon egy *loadPizzas* nevű függvényt. A metódus feladatai a következők:

- FETCH API használatával töltsse le a szerverről az összes pizza adatát (1p)
- Töltsse fel a pizzák adatait tartalmazó sorokkal a *pizza-list-table-body* elemet (nem kell a pizzák minden adatát megjelenítenie, de legyenek egyértelműen azonosíthatók!) (1p)

Hívja meg a függvényt, hogy betöltődjenek a táblázatba az adatok.

Az előbbi táblázat sorainak 'click' eseményére implementáljon egy *selectPizza* nevű függvényt. Ennek feladata, hogy testszöveges pizza kiválasztásának esetén a pizza adatait töltsse az oldal tetején található form-ba. (2p)

Implementáljon egy *getFormData* nevű függvényt, ami lekéri a formról az adatokat és egy pizza objektumot ad vissza. Biztosítsa, hogy az adatok típushelyesek legyenek. A későbbiek során ezt a függvényt használhatja az adatok begyűjtésére a kérések elküldése előtt. (1p)

Implementálja a *createPizza* nevű függvényt. Használjon FETCH API POST requestet. (2p)

Implementálja a *deletePizza* nevű függvényt. Kérje le a formtól a törölni kívánt pizza ID-ját, és küldje el a szerverre. Használjon FETCH API DELETE requestet. (1p)

Érje el, hogy a kliensen az egyes módosítások azonnal megjelenjenek a form tartalma pedig törlődjön (ehhez használhatja az előre megírt *resetForm* függvényt). (1p)

Mennyire találta nehéznek a feladatsort (1 - nagyon könnyű, 5 - nagyon nehéz)?